MERN Décalé – Séance 2

Persistance NoSQL avec MongoDB Atlas & Mongoose

Architecture ODM, Modélisation Relationnelle, Validations Avancées & Programmation Asynchrone

Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026–2027

Cadre Pédagogique du Format Décalé : Cette séance s'appuie directement sur l'architecture découplée **SoC (Router & Contrôleur)** mise en place lors de la Séance 1. Nous allons remplacer les tableaux temporaires en mémoire vive par une base de données cloud distribuée **MongoDB Atlas** via l'ODM **Mongoose**. **Validation 100% en séance :** Aucun compte-rendu lourd à rédiger à la maison ; les acquis sont validés en direct avec l'enseignant au fur et à mesure des **Points de contrôle**.

1 Pourquoi une Base de Données ? Du Volatile au Persistant

Lors de la Séance 1, nous avons créé des articles et des utilisateurs via Postman en les stockant dans des tableaux JavaScript locaux (`let articles = [...]`).

Prédisez. Si votre serveur Express est arrêté avec `Ctrl+C` dans le terminal puis relancé avec `npm run dev`, que renverra la requête `GET http://localhost:3000/api/articles` ?
1. La liste complète des articles saisis lors de vos tests précédents ?
2. Uniquement les deux articles écrits en dur au début de votre code source ?
3. Une erreur serveur ?

Constat technique : Les données saisies ont disparu. La mémoire vive (RAM) de Node.js est **volatile**. Dès l'arrêt du processus ou lors d'un déploiement en production, la mémoire est libérée.

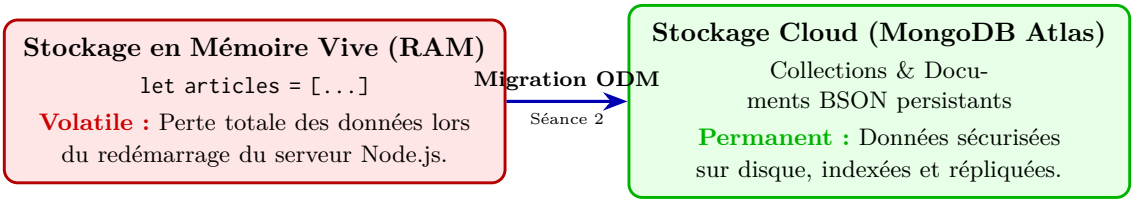


FIGURE 1 – Transition d'un stockage mémoire volatile vers la persistance cloud distribuée.

2 Mise en Place de MongoDB Atlas & Sécurisation des Secrets

MongoDB Atlas est une plateforme cloud hébergeant des bases NoSQL distribuées. En environnement professionnel, le serveur applicatif s'y connecte via une chaîne d'authentification sécurisée nommée **URI de connexion**.

Les 4 Étapes de Configuration sur Atlas

1. **Création du Cluster gratuit :** Rendez-vous directement sur `https://cloud.mongodb.com` (ou `https://www.mongodb.com/cloud/atlas/register`), créez un projet puis déployez un cluster partagé gratuit (**M0 Sandbox**) hébergé sur AWS (ex. région *Frankfurt*).
2. **Création de l'utilisateur de base de données :** Dans **Database Access**, créez un identifiant (ex : `admin_mern`) et un mot de passe robuste.
3. **Autorisation d'accès réseau :** Dans **Network Access**, ajoutez l'adresse `0.0.0.0/0` pour autoriser les requêtes issues du réseau de l'école.

4. **Récupération de l'URI :** Dans **Database** → **Connect** → **Drivers**, copiez l'URI au format :

```
mongodb+srv://admin_mern:<password>@cluster0.xxxxx.mongodb.net/blogDB?retryWrites=
    true&w=majority
```

Règle absolue de sécurité en entreprise : Ne commitez JAMAIS vos mots de passe et chaînes de connexion dans votre dépôt Git ! Le fichier `.env` contenant vos secrets doit figurer impérativement dans votre fichier `.gitignore`.

3 Architecture Mongoose : Schémas, Modèles et Relations NoSQL

Mongoose est un **ODM (Object Data Modeling)**. Il s'intercale entre votre code Express et MongoDB pour apporter structure, rigueur et validation à un univers NoSQL naturellement sans schéma (*schema-less*).

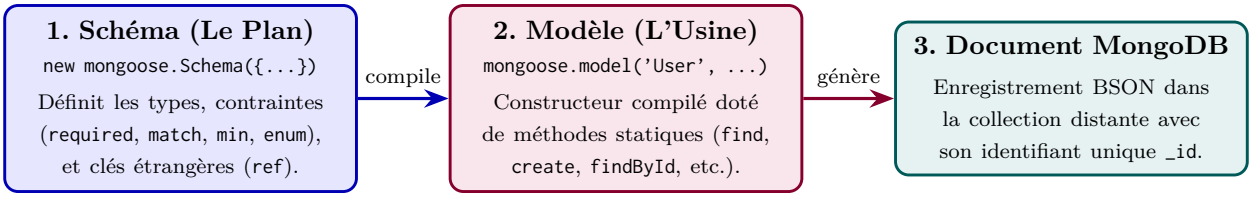


FIGURE 2 – Du Schéma Mongoose au Document dans MongoDB Atlas.

Exemple de Schéma avec Validation Regex, Contrainte d'Âge et Énumération

Voici le schéma `User` complet illustrant les validations d'entreprise :

1. **Vérification de forme d'email par Regex** avec `match` ;
2. **Contrainte numérique minimale sur l'âge** avec `min: 4` ;
3. **Énumération stricte** avec `enum` restreignant le champ `role` à `'user'` ou `'admin'`.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Le nom est obligatoire'], trim: true },
  email: {
    type: String,
    required: [true, "L'email est obligatoire"],
    unique: true,
    lowercase: true,
    trim: true,
    // Validation de forme d'email via Regex (match) :
    match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Format d'email invalide (ex: aya@polytechnique.
    tn)"]
  },
  age: {
    type: Number,
    // Age minimal autorise fixe a 4 ans :
    min: [4, "L'age minimum autorise est de 4 ans"],
    max: [120, "L'age maximum autorise est de 120 ans"]
  },
  role: {
    type: String,
    enum: {
      values: ['user', 'admin'],
      message: '{VALUE} n\'est pas un role autorise'
    },
    default: 'user'
  }
}, { timestamps: true });
```

```
module.exports = mongoose.model('User', userSchema);
```

Modélisation des Relations en NoSQL : 1 :1, 1 :N et N :N

Contrairement au modèle SQL fondé sur les jointures lourdes, MongoDB propose deux stratégies : l'**imbrication** (*embedding*) et le **référencement** (*referencing* avec `ObjectId`).

1. **Relation One-to-One (1 :1) :** Un utilisateur et son profil d'adresse.
   - *Approche imbriquée (performante) :* `profile: { bio: String, avatar: String }`
   - *Approche par référence :* `profile: { type: mongoose.Schema.Types.ObjectId, ref: 'Profile' }`
2. **Relation One-to-Many (1 :N) :** Un auteur (`User`) rédige plusieurs articles (`Article`). Dans chaque article, on stocke la clé primaire de son auteur via son `_id` :

```
// Dans models/articleModel.js
const articleSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
});
```

   Lors de la consultation, Mongoose reconstitue l'objet utilisateur complet à la volée avec `.populate()` :

```
const articles = await Article.find().populate('author', 'name email');
```

3. **Relation Many-to-Many (N :N) :** Un article possède plusieurs tags, et un tag qualifie plusieurs articles :

```
tags: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Tag' }] // Tableau de references
// Reconstitution dynamique :
const article = await Article.findById(id).populate('tags', 'label');
```

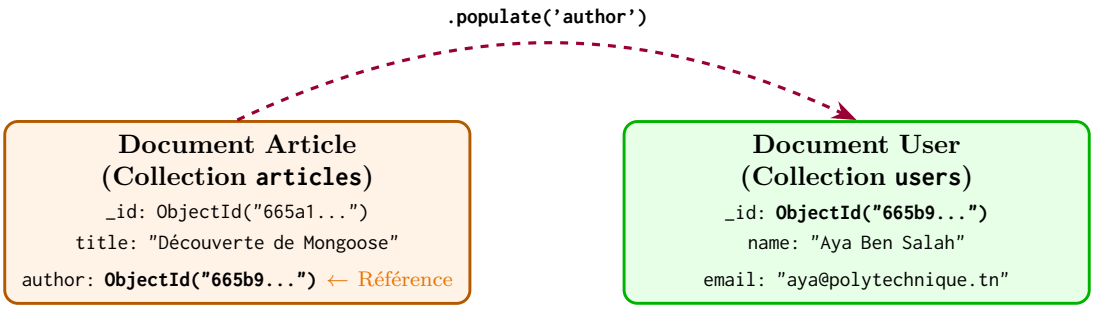


FIGURE 3 – Reconstitution dynamique de la relation One-to-Many via `.populate()`.

4 Le Paradigme Asynchrone : Des Promesses à `async/await`

Toute communication avec MongoDB Atlas nécessite un aller-retour réseau (I/O). Pour que le serveur Node.js ne soit jamais bloqué en attendant la base, Mongoose retourne des **Promesses (Promises)**.

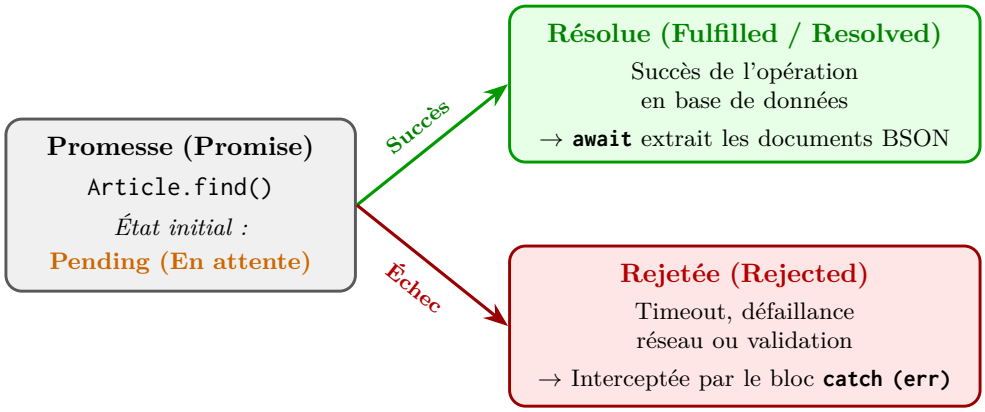


FIGURE 4 – Cycle de vie d'une Promesse et extraction synchrone via `async/await`.

Notion : `async/await` et les deux règles d'or. Le sucre syntaxique `async/await` permet d'écrire du code asynchrone avec la clarté d'un code synchrone :

1. Le mot-clé `await` ne peut être utilisé qu'**à l'intérieur** d'une fonction déclarée avec `async`.
2. `await` suspend temporairement l'exécution de la fonction locale en attendant la réponse de MongoDB, **sans jamais bloquer** le fil d'exécution de Node.js pour les autres clients.
3. Tout appel avec `await` doit être encapsulé dans un bloc `try { ... } catch (err) { res.status(500).json({ message: err.message }); }` pour intercepter les erreurs sans faire planter le serveur.

5 TP Guidé : Connexion et Migration Pas à Pas

Étape 1 : Installation des dépendances et fichier `.env`

Dans le terminal de votre projet, installez l'ODM Mongoose et la bibliothèque de gestion d'environnement `dotenv` :

```
npm install mongoose dotenv
```

Créez le fichier de configuration locale `.env` à la racine de votre projet : ➥ Fichier `.env` – lancer avec –

```
PORT=3000
MONGODB_URI=mongodb+srv://admin_mern:PolyMern2026!@cluster0.xxxxx.mongodb.net/blogDB?
    retryWrites=true&w=majority
```

Ajoutez immédiatement `.env` dans votre fichier `.gitignore` :

```
echo ".env" >> .gitignore
```

Étape 2 : Centraliser la connexion dans `src/config/db.js`

Créons un module dédié pour gérer l'ouverture de la connexion asynchrone à MongoDB :

➥ Fichier `src/config/db.js` – lancer avec –

```
const mongoose = require('mongoose');

async function connectDB(uri) {
  if (!uri) {
    throw new Error("L'URI MongoDB est manquant dans les variables d'environnement (.env)"
    );
  }
  await mongoose.connect(uri, {
    autoIndex: true // Génère automatiquement les index (ex: unique) en développement
  });
```

```
  console.log('Connexion a MongoDB Atlas reussie avec succes !');
}

module.exports = connectDB;
```

Étape 3 : Configurer et démarrer le serveur dans `src/server.js`

Comme en Séance 1, notre fichier `src/server.js` conserve la structure directe que vous connaissez : il instancie Express, monte le middleware JSON et les routeurs, puis démarre l'écoute après l'établissement de la connexion à MongoDB :

➥ Fichier `src/server.js` – lancer avec `npm run dev`

```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const articleRoutes = require('./routes/articleRoutes');
const userRoutes = require('./routes/userRoutes');

// Chargement des variables d'environnement (.env)
dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware pour parser le corps des requetes en JSON
app.use(express.json());

// Montage des routeurs modulaires
app.use('/api/articles', articleRoutes);
app.use('/api/users', userRoutes);

// Connexion asynchrone à MongoDB Atlas puis démarrage de l'écoute
(async () => {
  try {
    // 1. Établissement de la connexion distante à Atlas
    await connectDB(process.env.MONGODB_URI);

    // 2. Démarrage de l'écoute du serveur HTTP
    app.listen(PORT, () => {
      console.log(`Serveur MERN en ecoute sur http://localhost:${PORT}`);
    });
  } catch (err) {
    console.error('Erreur critique de demarrage du serveur :', err.message);
    process.exit(1);
  }
})();
```

Point de contrôle en séance. Lancez `npm run dev`. Le terminal doit afficher :
`Connexion a MongoDB Atlas reussie avec succes !`
`Serveur MERN en ecoute sur http://localhost:3000`
Si une erreur d'authentification s'affiche, vérifiez votre mot de passe et l'accès réseau `0.0.0.0/0` sur Atlas.

Étape 4 : Définition des Modèles Mongoose

Créons les deux modèles dans le dossier `src/models/`.

➥ Fichier `src/models/userModel.js` – lancer avec –

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
```

```
  {
    name: {
      type: String,
      required: [true, 'Le nom est obligatoire'],
      trim: true
    },
    email: {
      type: String,
      required: [true, "L'email est obligatoire"],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Format d'email invalide (ex:
    aya@polytechnique.tn)"]
    },
    age: {
      type: Number,
      min: [4, "L'age minimum autorise est de 4 ans"]
    },
    role: {
      type: String,
      enum: {
        values: ['user', 'admin'],
        message: '{VALUE} n\'est pas un role autorise'
      },
      default: 'user'
    }
  },
  { timestamps: true } // Injecte automatiquement createdAt et updatedAt
);

module.exports = mongoose.model('User', userSchema);
```

➥ Fichier `src/models/articleModel.js` – lancer avec –

```
const mongoose = require('mongoose');

const articleSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Le titre est obligatoire'],
      trim: true
    },
    content: {
      type: String,
      required: [true, 'Le contenu est obligatoire']
    },
    author: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, "L'auteur (authorId) est obligatoire"]
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model('Article', articleSchema);
```

Étape 5 : Migration Asynchrone des Contrôleurs

Remplaçons les tableaux en mémoire par les requêtes Mongoose asynchrones avec `async/await`.

➥ Fichier `src/controllers/userController.js` – lancer avec –

```
const User = require('../models/userModel');
```

```
// 1. Créer un nouvel utilisateur (POST /api/users)
const createUser = async (req, res) => {
  try {
    const { name, email, age, role } = req.body;
    if (!name || !email) {
      return res.status(400).json({ message: 'Le nom et l\'email sont requis' });
    }
    // Vérification de doublon d'email
    const exists = await User.findOne({ email });
    if (exists) {
      return res.status(409).json({ message: 'Cet email est deja utilise' });
    }
    const user = await User.create({ name, email, age, role });
    res.status(201).json(user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 2. Obtenir tous les utilisateurs (GET /api/users)
const getUsers = async (req, res) => {
  try {
    const users = await User.find();
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 3. Obtenir un utilisateur par son identifiant (GET /api/users/:id)
const getUserById = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non trouve' });
    }
    res.status(200).json(user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

module.exports = { createUser, getUsers, getUserById };
```

➥ Fichier src/controllers/articleController.js – lancer avec –

```
const Article = require('../models/articleModel');
const User = require('../models/userModel');

// 1. Récupérer tous les articles avec les données complètes de l'auteur
const getAllArticles = async (req, res) => {
  try {
    const articles = await Article.find().populate('author', 'name email');
    res.status(200).json(articles);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 2. Créer un nouvel article lié à un auteur
const createArticle = async (req, res) => {
  try {
    const { title, content, authorId } = req.body;
    if (!title || !content || !authorId) {
      return res.status(400).json({ message: 'title, content et authorId sont obligatoires
    ' });
```

```
    }
    // Vérifier l'existence réelle de l'auteur dans la collection 'users'
    const author = await User.findById(authorId);
    if (!author) {
      return res.status(404).json({ message: 'Auteur non trouve (authorId invalide)' });
    }

    const newArticle = await Article.create({ title, content, author: authorId });
    const populatedArticle = await newArticle.populate('author', 'name email');
    res.status(201).json(populatedArticle);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 3. Récupérer un article unique par son ID
const getArticleById = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id).populate('author', 'name email')
    ;
    if (!article) {
      return res.status(404).json({ message: 'Article non trouve' });
    }
    res.status(200).json(article);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 4. Mettre à jour un article (PUT /api/articles/:id)
const updateArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!article) {
      return res.status(404).json({ message: 'Article introuvable pour la mise a jour' });
    }
    const populated = await article.populate('author', 'name email');
    res.status(200).json(populated);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 5. Supprimer un article (DELETE /api/articles/:id)
const deleteArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndDelete(req.params.id);
    if (!article) {
      return res.status(404).json({ message: 'Article introuvable pour la suppression' });
    }
    res.status(200).json({ message: 'Article supprime avec succes', id: req.params.id });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

module.exports = { getAllArticles, createArticle, getArticleById, updateArticle,
    deleteArticle };
```

Étape 6 : Preuve de la Puissance du Découplage SoC

Observez vos fichiers `routes/articleRoutes.js` et `routes/userRoutes.js` : ils n'ont subi **strictement aucune modification** ! Les routes associent toujours les chemins aux contrôleurs. Ce découplage garanti dès la Séance 1 permet de changer de moteur de stockage sans casser l'API.

Étape 7 : Validation Complète de Bout en Bout avec Postman

Exécutez ce scénario méthodique de test dans Postman :

1. **Création d'un utilisateur valide (POST `http://localhost:3000/api/users`) :**

```
{
  "name": "Aya Ben Salah",
  "email": "aya@polytechnique.tn",
  "age": 22,
  "role": "admin"
}
```

   → Statut **201 Created**. **Copiez la valeur du champ `_id`** généré par MongoDB (ex : `"665b9..."`).
2. **Test de validation des contraintes (rejet par Regex et âge minimum) :** Tentez d'insérer un document non conforme avec un format d'email erroné et un âge inférieur à 4 ans :

```
{
  "name": "Test Invalide",
  "email": "adresse_sans_arobase",
  "age": 2
}
```

   → Statut **400 Bad Request** ! Mongoose bloque l'insertion sans polluer la base de données.
3. **Création d'un article lié (POST `http://localhost:3000/api/articles`) :** Collez l'`_id` de l'utilisateur dans le champ `authorId` :

```
{
  "title": "Persistance NoSQL avec Mongoose",
  "content": "L'architecture SoC garantit une transition fluide vers MongoDB Atlas.",
  "authorId": "COLLEZ_ICI_L_ID_UTILISATEUR"
}
```

   → Statut **201 Created**. Le champ `author` contient directement le profil complet grâce à `.populate()` !
4. **Vérification de la persistance réelle :** Arrêtez votre serveur dans le terminal avec `Ctrl+C`, puis relancez-le avec `npm run dev`. Exécutez un `GET http://localhost:3000/api/articles` : vos données sont toujours intactes, directement lues depuis MongoDB Atlas !

Point de contrôle en séance. Montrez à l'enseignant votre collection `blogDB` dans l'interface de MongoDB Atlas et la requête Postman affichant l'article avec l'auteur peuplé (`.populate()`).

6 Atelier Pratique en Autonomie (En Séance)

Mission 1 : Intégrité Référentielle lors de la Suppression d'un Utilisateur

Dans `src/controllers/userController.js`, ajoutez la fonction `deleteUser` et exposez-la sur `DELETE /api/users/:id` dans `routes/userRoutes.js` :

1. Avant de supprimer l'utilisateur, vérifiez l'existence d'articles liés :
   `const hasArticles = await Article.findOne({ author: req.params.id });`
2. Si l'utilisateur possède au moins un article, refusez formellement la suppression :

Renvoyez le statut **400 Bad Request** avec : `{ "message": "Impossible de supprimer un auteur ayant des articles publies" }`.
3. Sinon, procédez à la suppression avec `User.findByIdAndDelete(req.params.id)` et renvoyez le statut **200 OK**.

Mission 2 : Gestion Professionnelle des Identifiants Invalides (`CastError`)

Dans MongoDB, les identifiants `_id` sont des chaînes hexadécimales de 24 caractères. Si un client envoie un identifiant mal formé (ex : `GET /api/articles/abc`), Mongoose lève une exception `CastError` interceptée dans le bloc `catch (err)`.

1. Dans vos fonctions de contrôleur recevant un identifiant (`getArticleById`, `getUserById`), examinez le bloc `catch (err)`.
2. Interceptez spécifiquement l'erreur si `err.name === 'CastError'`.
3. Renvoyez immédiatement le statut **400 Bad Request** avec : `{ "message": "Identifiant de ressource invalide (format ObjectId attendu)" }` ; sinon renvoyez le statut **500**.
4. Testez avec Postman en appelant `GET /api/articles/abc` et observez la réponse 400 propre.

Point de contrôle en séance. Faites valider la Mission 1 et la Mission 2 directement par l'enseignant sur votre poste de travail. Cette validation en direct clôture vos objectifs de la séance !

7 Récapitulatif

Six idées fondamentales à retenir

1. La mémoire vive (RAM) est éphémère ; **MongoDB Atlas** assure la persistance, l'intégrité et la scalabilité des données dans le cloud.
2. Toute opération de base de données est une entrée/sortie réseau asynchrone retournant une **Promesse**, manipulée proprement avec le duo **`async / await`**.
3. Les fonctions asynchrones doivent être protégées par des blocs **`try ... catch`** avec retour d'un statut HTTP d'erreur (`res.status(500).json(...)`) pour éviter le plantage du serveur.
4. **Mongoose** structure l'univers NoSQL grâce aux **Schémas** (règles et types de validation) compilés en **Modèles** (constructeurs dotés de méthodes CRUD).
5. Les relations NoSQL s'expriment par des références d'identifiants (`type: ObjectId, ref: 'User'`) et sont résolues dynamiquement grâce à **`.populate()`**.
6. Les secrets d'accès à la base de données ne doivent jamais figurer dans le code source : on utilise le module **`dotenv`** et le fichier **`.env`** exclu par **`.gitignore`**.

Je veux...	J'écris...
Connecter mon serveur à MongoDB Atlas	`await mongoose.connect(process.env.MONGODB_URI);`
Définir un champ texte avec regex d'email	`email: { type: String, match: [/regex/, 'msg'] }`
Définir une référence relationnelle vers User	`author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }`
Récupérer les documents avec jointure virtuelle	`await Article.find().populate('author', 'name email');`
Trouver un document par son identifiant	`const article = await Article.findById(req.params.id);`
Créer et insérer un document validé	`const newDoc = await Article.create(req.body);`
Mettre à jour avec exécution des validateurs	`await Article.findByIdAndUpdate(id, data, { new: true });`
Supprimer définitivement un document	`await Article.findByIdAndDelete(req.params.id);`

Je sais faire (auto-évaluation)

☐	Expliquer la différence entre mémoire vive volatile et stockage persistant	Section 1
☐	Déployer un cluster MongoDB Atlas et sécuriser l'URI avec `dotenv`	Section 2
☐	Définir un schéma Mongoose avec contraintes Regex, âge minimum et énumération	Section 3
☐	Modéliser une relation 1 :N et reconstituer les données via `.populate()`	Section 3
☐	Écrire des contrôleurs asynchrones avec `async/await` et gestion d'erreurs	Section 5
☐	Protéger la suppression contre la corruption relationnelle	Section 6
☐	Intercepter les erreurs de format d'identifiant (`CastError`)	Section 6

8 Références des Notions de la Séance (Session Concept Reference)

1. Méthodes Indispensables du Modèle Mongoose

Méthode Mongoose	Rôle dans l'API REST	Exemple d'écriture
`Model.find(filtre)`	Récupère une liste de documents (tous si filtre vide)	`await Article.find({ author: id });`
`Model.findById(id)`	Récupère un document unique par sa clé primaire `_id`	`await Article.findById(id);`
`Model.findOne(filtre)`	Récupère le premier document validant le critère	`await User.findOne({ email });`
`Model.create(donnees)`	Valide et insère un ou plusieurs nouveaux documents	`await User.create(req.body);`
`Model.findByIdAndUpdate`	Modifie un document existant (`opt: { new: true }`)	`await Article.findByIdAndUpdate(id, data, { new: true });`
`Model.findByIdAndDelete`	Supprime le document désigné par son identifiant	`await Article.findByIdAndDelete(id);`
`query.populate('champ')`	Remplace un `ObjectId` par le document lié	`await Article.find().populate('author', 'name email');`
`Model.countDocuments()`	Dénombre efficacement les documents d'une collection	`await Article.countDocuments();`

2. Programmation Asynchrone en Node.js

Concept / Syntaxe	Signification technique	Exemple d'usage
`async function`	Déclare une fonction asynchrone qui renvoie une Promesse	`const getAll = async (req, res) => { ... };`
`await expression`	Met en pause l'exécution locale jusqu'à résolution	`const data = await Model.find();`
`try { ... } catch (err)`	Capture toute erreur asynchrone (évite le crash du serveur)	`try { ... } catch (err) { res.status(500).json(...) }`
`Promise.all([p1, p2])`	Exécute plusieurs Promesses en parallèle et attend la fin	`const [arts, users] = await Promise.all([...]);`
`new Promise((res, rej))`	Crée une Promesse personnalisée avec gestion succès/échec	`return new Promise((resolve, reject) => ...);`

3. Types et Validateurs de Schéma Mongoose

Option de Schéma	Rôle de validation	Exemple concret
`type: String / Number`	Typage strict des propriétés du document	`title: { type: String }`
`type: ObjectId, ref`	Définition d'une clé étrangère vers une autre collection	`author: { type: ObjectId, ref: 'User' }`
`required: [true, '...']`	Rend la présence du champ obligatoire à l'insertion	`email: { required: [true, 'Email requis'] }`
`unique: true`	Crée un index d'unicité dans MongoDB (rejette doublons)	`email: { type: String, unique: true }`
`match: [/regex/, '...']`	Valide le format textuel par une expression régulière	`email: { match: [/^\S+@\S+\.\S+$/] }`
`min: [4, '...'] / max`	Impose des bornes numériques minimales et maximales	`age: { type: Number, min: 4 }`
`trim: true`	Supprime automatiquement les espaces en début et fin	`name: { type: String, trim: true }`
`lowercase: true`	Convertit la chaîne en minuscules avant enregistrement	`email: { type: String, lowercase: true }`
`enum: ['user', 'admin']`	Restreint la valeur à un ensemble fini de chaînes	`role: { type: String, enum: ['user', 'admin'] }`
`default: valeur`	Attribue une valeur par défaut si le champ est omis	`role: { type: String, default: 'user' }`
`{ timestamps: true }`	Crée automatiquement les champs `createdAt` & `updatedAt`	`new mongoose.Schema({ ... }, { timestamps: true });`

4. Expressions Régulières (Regex) pour la Validation d'API

Symbole / Motif	**Signification & Fonctionnement**	**Exemple d'Application API**
`^` et `$`	Début (`^`) et fin (`$`) stricts de chaîne	`/^aya$/` correspond uniquement à `"aya"`
`[a-zA-Z0-9]`	Classe de caractères autorisés	Identifiant : `/^[a-zA-Z0-9_-]{3,16}$/`
`[^...]`	Négation : exclut les caractères listés	`[^\s@]+` (aucun espace ni arobase)
`\d`	Chiffre décimal équivalent à `[0-9]`	Téléphone : `/^\d{8}$/` (8 chiffres)
`\w`	Caractère de mot (`[a-zA-Z0-9_]`)	Slug d'article : `/^[\w-]+$/`
`\s` / `\S`	Espace blanc / Non-espace blanc	Mot de passe sans espace : `/^\S+$/`
`+` / `*` / `?`	Quantificateurs : 1 ou +, 0 ou +, 0 ou 1	Indicatif optionnel : `/(\+216)?\d{8}/`
`{n,m}`	Répétition bornée (entre n et m fois)	Mot de passe : `/^\S{8,32}$/`
Motif Email	Structure standard `identifiant@domaine.ext`	`/^\S+@\S+\.\S+$/`
`match: [/reg/, '...']`	Propriété Mongoose exécutant le regex	Déclenche `ValidationError` (400) si non conforme

5. Configuration d'Environnement et Sécurité (`dotenv`)

Élément	Rôle dans l'application	Bonne pratique professionnelle
`dotenv.config()`	Charge les variables du fichier `.env` dans `process.env`	À appeler au tout début de `server.js` avant tout code
`process.env.NOM`	Accède en lecture à une variable d'environnement	`const PORT = process.env.PORT || 3000;`
Fichier `.env`	Stocke localement les secrets (mots de passe, clés, URI)	Ne jamais commiter dans Git (à placer dans `.gitignore`)
Fichier `.env.example`	Modèle documenté sans aucun mot de passe secret	Exemple : `MONGODB_URI=mongodb+srv://...`
URI `mongodb+srv://`	Chaîne de connexion et routage DNS vers le cluster Atlas	Toujours vérifier l'URL-encoding des caractères du mot de passe

La prochaine fois (Séance 3 Décalée : Sécurisation de l'API avec JWT & Authentification). Maintenant que nos utilisateurs et articles sont stockés de façon permanente et relationnelle dans MongoDB, nous allons protéger les routes de notre API. Nous aborderons le hachage sécurisé des mots de passe avec **bcrypt**, la génération de jetons d'authentification **JSON Web Tokens (JWT)** et la création de **middlewares d'authentification** pour restreindre la création et la modification d'articles aux seuls utilisateurs connectés !